

What Does LoRA Change?

An interpretability study of Low-Rank Adaptation in a small LLM

Marcos Lahoz

Abstract

Low-Rank Adaptation (LoRA) fine-tunes large language models by learning small low-rank updates to frozen weight matrices. It is ubiquitous, yet it is rarely asked *what* a LoRA adapter actually changes inside the model. We study this on **Qwen2.5-1.5B** instruction-tuned on Alpaca, with two complementary, training-free diagnostics: (i) the *relative update magnitude* $\|\Delta W\|/\|W\|$ and the effective rank of ΔW for every adapted projection, revealing where LoRA concentrates its capacity; and (ii) the *representation drift* between the adapter-enabled and adapter-disabled model, layer by layer, revealing how far the internal representations move and at what depth. We run these across an ablation over the LoRA rank and the set of adapted modules. We then test whether the findings *generalise*: we repeat the diagnostics on a second model family (**SmolLM2-1.7B**) and a second PEFT variant (DoRA), with two upgrades — *exact* update norms measured by merging the adapter and diffing against the base weights, and a *behavioural validation* via held-out instruction loss. The first three findings replicate. The fourth becomes a sharper, DoRA-specific result: reading DoRA’s learned magnitude vector into the exact update shows that DoRA matches LoRA in update *magnitude* and in its effect on representations, yet writes that change into a far *higher-rank* weight perturbation (participation ratio $\approx 540/1281$ vs. $\approx 13/10$ for LoRA). The magnitude component, not the direction, is what distinguishes DoRA in weight space. The framework turns “LoRA works” into measurable, localised statements about a fine-tuned model.

1 Introduction

LoRA [1] freezes a pretrained weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and learns a low-rank update $\Delta W = \frac{\alpha}{r}BA$ with $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $r \ll d$. It is the default way to fine-tune LLMs cheaply, but it is usually treated as a black box. We ask three questions:

1. **Where** does LoRA put its capacity — which layers and which projection types (q, k, v, o vs. the MLP gate, up, down) receive the largest updates?
2. **How much** do the model’s hidden representations actually move once the adapter is enabled, and at which depth?
3. How do (1) and (2) depend on the LoRA **rank** and the set of **adapted modules**?

The analyses are training-free post-hoc diagnostics and apply to any LoRA-adapted transformer.

2 Method

(1) **Effective update magnitude.** For each adapted module we form the explicit update $\Delta W = \frac{\alpha}{r}BA$ and report the relative Frobenius norm

$$\rho = \frac{\|\Delta W\|_F}{\|W\|_F},$$

i.e. how large the change is compared with the original weight. We also summarise the *shape* of ΔW by the participation ratio of its singular values $\{\sigma_i\}$,

$$\text{eff-rank}(\Delta W) = \exp\left(-\sum_i p_i \log p_i\right), \quad p_i = \frac{\sigma_i}{\sum_j \sigma_j},$$

which measures how many directions the update effectively uses. For a low-rank $\Delta W = \frac{\alpha}{r}BA$ this is bounded by r ; for the *exact* merged update of DoRA (§5) it need not be, since the magnitude rescaling acts on the full-rank base weight.

(2) Representation drift. Let $h_{\text{on}}^{(\ell)}(x)$ and $h_{\text{off}}^{(\ell)}(x)$ be the mean-pooled hidden state at layer ℓ for input x with the adapter enabled and disabled (the rest of the model is identical). Over a held-out set $\mathcal{X}$ we report, per layer, the mean cosine similarity and the relative L_2 drift

$$\cos^{(\ell)} = \frac{1}{|\mathcal{X}|} \sum_x \cos(h_{\text{on}}^{(\ell)}, h_{\text{off}}^{(\ell)}), \quad \delta^{(\ell)} = \frac{1}{|\mathcal{X}|} \sum_x \frac{\|h_{\text{on}}^{(\ell)} - h_{\text{off}}^{(\ell)}\|}{\|h_{\text{off}}^{(\ell)}\|}.$$

This isolates the causal effect of the adapter on the representations.

3 Experimental setup

We fine-tune the base **Qwen2.5-1.5B** on a subset of Alpaca with an instruction template, training only the LoRA adapters. We ablate the rank $r \in \{4, 16, 64\}$ and the adapted module set (*attn*: q, k, v, o ; *all*: +gate, up, down). Representation drift is measured on a fixed set of held-out instructions. All code, the ablation runner and the plots are in the repository.

4 Results

We fine-tune for one epoch on 2,000 Alpaca examples per configuration. Training converges to a similar loss everywhere (1.39–1.47), with only small gains from larger rank or adapting all modules (best: rank 64, all, train loss 1.41), already hinting at *diminishing returns*. Table 1 and Figure 1 summarise the diagnostics.

rank	modules	mean ρ	$\rho(\text{attn})$	$\rho(\text{mlp})$	eff-rank(ΔW)	drift (rel. L_2)
4	attn	0.0063	0.0063	–	3.24	0.299
16	attn	0.0088	0.0088	–	11.77	0.320
64	attn	0.0145	0.0145	–	44.90	0.314
4	all	0.0039	0.0042	0.0035	3.55	0.250
16	all	0.0063	0.0069	0.0056	13.41	0.257
64	all	0.0117	0.0127	0.0104	52.01	0.224

Table 1: Ablation summary. $\rho = \|\Delta W\|/\|W\|$ averaged over adapted modules; eff-rank is the participation ratio of ΔW ’s singular values; drift is the final-layer relative- L_2 representation drift.

Four findings stand out:

1. **Updates are tiny in magnitude.** ρ stays around 0.4%–1.5%: LoRA makes small, targeted edits to the weights rather than large changes.
2. **LoRA uses (almost) its full rank budget.** The effective rank of ΔW tracks r closely (3.2 at $r=4$, 11.8 at $r=16$, ~ 45 –52 at $r=64$): the adaptation is genuinely high-dimensional, not a rank-1 shortcut.

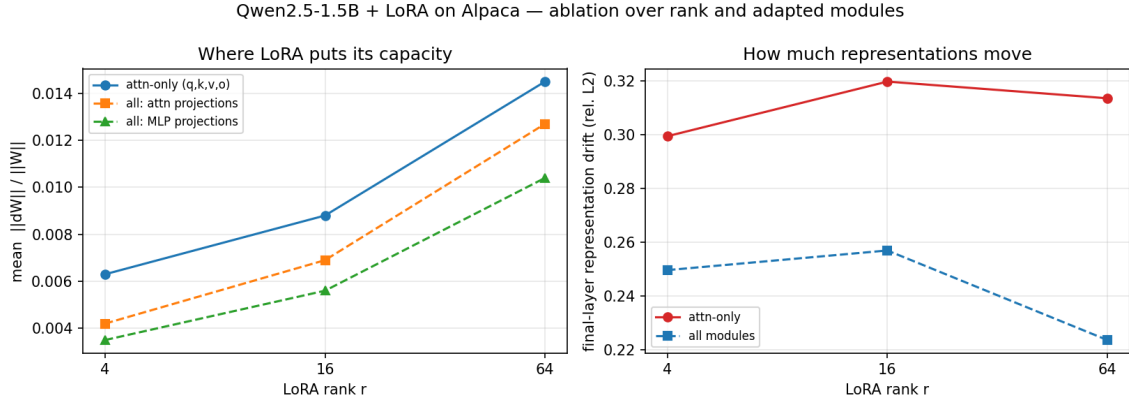


Figure 1: Left: relative update magnitude grows with rank, and attention projections receive larger updates than the MLP. Right: final-layer representation drift; adapting all modules moves representations less than attention-only at matched rank.

3. **Attention changes more than the MLP.** Under the *all* setting, attention projections consistently receive larger relative updates than the MLP across every rank.
4. **Fine-tuning rescales rather than rotates representations.** Final-layer cosine similarity between the adapter-on and adapter-off model stays high (0.96–0.98) while the relative- L_2 drift is sizeable (0.22–0.32): the *direction* of the representations is largely preserved, but their *magnitude/offset* moves. Spreading capacity across all modules yields lower per-layer drift than attention-only at the same rank.

Together with the near-flat training loss across ranks, this indicates that for this model/task a small rank already captures most of the adaptation, and the extra capacity of large ranks is only partly used.

5 Generality: a second model family and DoRA

The ablation above is a single-model, single-variant study. To test whether its conclusions are properties of *LoRA fine-tuning* rather than of *Qwen2.5-1.5B*, we repeat the diagnostics at rank 16 (*all* modules, same data and budget) along two new axes: a second model family, *SmolLM2-1.7B* (a Llama-style architecture), and a second PEFT variant, **DoRA** [2], which decomposes the update into a magnitude and a direction component.

Two methodological upgrades accompany the extension. First, update norms are now *exact*: we snapshot the frozen base weights, merge the trained adapter, and measure ΔW as the difference of the merged and base matrices — for DoRA, the (α/r) BA formula would miss the magnitude component, whereas the merge-and-diff measurement includes it. Second, every configuration passes a *behavioural validation* before its weights are interpreted: mean token-level cross-entropy on 200 held-out Alpaca examples, base vs. fine-tuned, so the diagnostics are guaranteed to describe runs that actually adapted.

The first three findings replicate on the second family (Table 2): updates remain tiny (mean $\rho \leq 0.7\%$ for both LoRA and DoRA; SmolLM2 is even more frugal at ≈ 0.2 – 0.3%); for LoRA most of the rank budget is used (13.4/16 on Qwen, 10.2/16 on SmolLM2 — substantial but model-dependent); attention receives larger relative updates than the MLP in all four configurations; and the rescale-not-rotate signature persists (final-layer cosine 0.97/0.93 with sizeable relative- L_2 drift 0.26/0.37 — SmolLM2 moves direction somewhat more than Qwen). LoRA and DoRA drift almost identically: their *behavioural* effect is the same.

model	variant	loss (base→ft)	mean ρ	$\rho(\text{attn})$	$\rho(\text{mlp})$	eff-rank	drift cos	drift L_2
Qwen2.5-1.5B	LoRA	1.97 → 1.45	0.0063	0.0069	0.0056	13.4	0.971	0.257
Qwen2.5-1.5B	DoRA	1.97 → 1.45	0.0066	0.0071	0.0060	540.5	0.971	0.259
SmolLM2-1.7B	LoRA	2.23 → 1.37	0.0022	0.0023	0.0020	10.2	0.927	0.375
SmolLM2-1.7B	DoRA	2.23 → 1.37	0.0028	0.0029	0.0027	1280.9	0.928	0.373

Table 2: Generality study (rank 16, all modules, 2,000 Alpaca examples, one epoch). Update norms are exact (merge-and-diff), reading DoRA’s learned magnitude vector so its magnitude component is included in ΔW . Every configuration improves held-out instruction loss by 0.5–0.9 nats before being interpreted.

A new finding: DoRA differs from LoRA in the rank of the update, not its magnitude. Update magnitude, attention bias and representation drift all match between LoRA and DoRA within noise — but the *effective rank* of the merged update does not. For LoRA the exact ΔW is genuinely low-rank, tracking the budget r (participation ratio 13.4 on Qwen, 10.2 on SmolLM2). For DoRA the exact ΔW is *high-rank* — ≈ 540 on Qwen and ≈ 1281 on SmolLM2 — because the per-row magnitude rescaling $m \cdot (W_0 + \Delta V) / \|W_0 + \Delta V\|_{\text{row}}$ acts on the full-rank base weight, spreading ΔW across hundreds of directions even though DoRA’s *trainable* directional component is still rank $\leq r$. This difference is invisible to the directional (α/r) BA estimator and appears only once the magnitude component is measured. So at this scale and task LoRA and DoRA are indistinguishable in update magnitude and in their effect on representations, yet DoRA writes that same change into a much higher-rank weight perturbation. Whether this higher-rank footprint translates into a behavioural advantage at larger scale or on tasks where DoRA reports gains is an open question the framework can answer directly.

6 Discussion and future work

The two diagnostics localise fine-tuning: ρ shows which weights move, the effective rank shows how low-dimensional the change is, and the drift curve shows where in the network behaviour is reshaped (typically the later layers). The generality study (Section 5) shows these are statements about LoRA-style fine-tuning, not about one model, and that DoRA leaves update magnitude and representation drift unchanged while writing a far higher-rank weight perturbation — a difference only the exact, magnitude-aware diagnostic reveals. Natural extensions: (i) scaling the grid to 7B+ models and longer training, where DoRA’s reported gains might translate into diagnostic differences; (ii) trained linear *probes* on the representations before/after, connecting to structural-probe methodology; (iii) tracking weight-space directions of the merged updates across tasks.

References

- [1] Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
- [2] Liu, S.-Y. et al. (2024). *DoRA: Weight-Decomposed Low-Rank Adaptation*. ICML.
- [3] Hewitt, J. & Manning, C. (2019). *A Structural Probe for Finding Syntax in Word Representations*. NAACL.
- [4] Qwen Team (2024). *Qwen2.5 Technical Report*.